Practice Block II

Marc Mateu, Luis Calvet

November 2025

Table of Contents

1 Introduction

1.1 Brief description of the problem

The problem addressed in this assignment is the **Frozen Lake** game. The environment consists of an $n \times n$ grid board where the agent must navigate to find a path from its starting position to a goal tile.

The map features dangerous tiles: holes. If the agent steps into one of these holes during the journey, it loses the game.

The main characteristic defining the complexity of this problem is the stochasticity of movement. Since the terrain is slippery, the agent does not always move in the chosen direction. To guarantee this condition, the *Gymnasium* environment is used with the mandatory parameter `is_slippery=True` .

1.2 Objective of the assignment

The main objective of this assignment is to apply the AI concepts and techniques learned during the course to design and develop agents capable of solving the presented problem.

Specifically, the task focuses on the implementation of reinforcement learning algorithms and genetic algorithms.

Beyond the technical implementation, the assignment aims to conduct a deep analysis of the solutions. This includes exposing and justifying the design decisions made for each algorithm, as well as studying the effect that different parameters have on the final solution.

Finally, it seeks to compare the performances obtained between the different techniques and analyze the resulting policies, relating the observed behavior to the theory seen in class.

1.3 Implemented algorithms

To solve this problem, four different algorithms have been selected and implemented, covering both reinforcement learning and evolutionary computation techniques. According to the assignment's requirements, the developed algorithms are:

- **SARSA (State-Action-Reward-State-Action):** An *on-policy* reinforcement learning method based on Temporal Difference (TD).
- **Q-Learning:** An *off-policy* reinforcement learning method, also based on TD, which seeks to directly learn the optimal policy regardless of the exploratory actions taken.
- **Monte Carlo:** A method based on full episodic experience, updating state values based on the average returns observed at the end of each episode.

- **Genetic Algorithm:** An evolutionary computation-based approach, where a population of agents evolves over generations through selection, crossover, and mutation processes to find the optimal sequence of actions.

1.4 Justification for the non-implemented algorithm

Among the algorithms covered in class, it was decided to discard the implementation of techniques based on **Dynamic Programming**.

The main reason for this decision is that Dynamic Programming methods require a complete and perfect model of the environment (i.e., knowing a priori the transition probabilities $P(s'|s,a)$ and the reward function). The goal of this assignment is to simulate a realistic scenario where the agent does not know the physical rules of the world. Therefore, priority was given to *model-free* algorithms (such as Q-Learning, SARSA, and Monte Carlo), which force the agent to learn the optimal policy exclusively through experience, exploration, and direct interaction with the game, without access to the underlying transition matrix.

As stated, Dynamic Programming algorithms are based on the iterative updating of state values using the Bellman equation. The mathematical formula governing these updates is as follows:

$$V_{k+1}(s) = \max_a \sum_{s'} P(s' \mid s,a) \left[R(s,a,s') + \gamma V_k(s')\right] \tag{1}$$

Where:

- $P(s' \mid s,a)$ is the transition probability to state s' given state s and action a.
- $R(s,a,s')$ is the reward obtained.
- γ is the discount factor.

As can be explicitly seen in the formula, the term $P(s' \mid s,a)$ is indispensable for the calculation. Since in this problem the agent does not have access to the environment's transition matrix (it does not know *a priori* where the slippery ice will take it), direct application of this formula is unfeasible without a prior model learning phase.

2 Environment Description

2.1 How does FrozenLake work?

FrozenLake is a discrete simulation environment representing a frozen lake in the form of an $n \times n$ *gridworld*. The fundamental objective of the game is for the agent to find a **safe path** from its starting position to the goal tile.

The map is composed of four types of tiles, represented by letters, each with a different effect on the game:

- **S (Start):** Indicates the starting position where the agent begins (usually the top-left tile).
- **H (Hole):** Hole in the ice. These are the tiles to avoid; if the agent steps on one, it falls into the water and the episode ends with a zero reward (defeat).
- **G (Goal):** The goal tile. Reaching here means successfully completing the episode and receiving the reward (victory).

At each time step, the agent must choose a movement action among four possibilities: 0: Left, 1: Down, 2: Right, 3: Up.

2.2 Role of `is_slippery=True`

The `is_slippery=True` parameter is the determining factor that converts FrozenLake from a trivial pathfinding problem into a complex Reinforcement Learning problem.

When this parameter is enabled, the environment becomes **stochastic**. This means state transitions are non-deterministic, as discussed in the non-implementation of Dynamic Programming: the action chosen by the agent does not guarantee a unique outcome. If this parameter were `False`, the agent would always move exactly where it decided, and the problem could be solved with classical search algorithms (like A* or BFS solved in the previous assignment) without the need for learning. Activating this mode forces the agent to learn robust policies that minimize risk, rather than simply memorizing a path.

2.3 Explanation of the slippery terrain

The concept of "slippery terrain" refers to the uncertainty in movement. Due to the ice, the agent can slip and move in a direction perpendicular to the desired one. In the standard FrozenLake configuration, when the agent chooses an action (for example, moving to the **Right**), there is a probability distribution over the actual outcome:

- **33.3% probability** of effectively moving to the **Right**.
- **33.3% probability** of slipping and moving **Up** (perpendicular).
- **33.3% probability** of slipping and moving **Down** (perpendicular).

This dynamic implies that the agent never has total control over its movement. Therefore, an optimal policy in this environment is not necessarily the shortest route, but the safest route (one that, even if the agent slips, prevents it from falling into a hole).

3 General Methodology

3.1 How are the agents trained?

The agent training process follows the classic Agent-Environment interaction scheme defined in the Reinforcement Learning framework. Training is organized into **episodes**. An episode begins with the agent at the start tile (S) and ends when it reaches the goal (G) or falls into a hole (H).

During each time step t, the following cycle occurs:

1. The agent observes the current state S_t.
2. The agent selects an action A_t according to its current policy (for example, ε-greedy).
3. The environment executes the action and returns a reward R_{t+1} and the new state S_{t+1}.
4. The agent uses this information to update its knowledge (the Q-Table in the case of Q-Learning/SARSA or the genome in the case of the Genetic algorithm).

This cycle is repeated until a stopping criterion is met, which in this assignment has been defined as a maximum number of episodes or the convergence of the policy.

3.1.1 Executions for reliable results

One of the main challenges of this assignment is the stochasticity introduced by the `is_slippery=True` parameter. Because of this, a single training run or a single agent evaluation is not representative; an agent could get lucky and reach the goal by chance, or get unlucky and slip into a hole despite making the right decision.

To mitigate this effect and obtain valid results, the following methodology was adopted:

- **Repetition of experiments:** Each algorithm has been executed multiple times with different random seeds.
- **Alteration of hyperparameters:** The values of different hyperparameters have been changed during the various executions to observe how the corresponding agent behaves and to see if its behavior is reliable and consistent with the hyperparameters.
- **Sliding window:** To better visualize the reward evolution, a moving average over the last 100 episodes has been applied.

3.2 Evaluation metrics

To compare the performance of the different algorithms, the metrics suggested in the assignment description have been used, allowing evaluation of both the effectiveness and efficiency of the solution:

- **Success Rate:** The percentage of episodes in which the agent successfully reaches the goal tile. Given the difficulty of the slippery map, this is the primary metric.
- **Accumulated reward:** The sum of rewards obtained throughout the episodes. Since FrozenLake only gives 1 point for winning and 0 otherwise, this metric is directly correlated with the success rate.
- **Execution time:** The computation time required to complete the training, useful for comparing the computational cost between simple methods (SARSA) and evolutionary ones (Genetic).

4 Implemented algorithms

4.1 SARSA

4.1.1 Design decision

The chosen design implements SARSA as an on-policy method, where the agent updates the Q-table using the action actually executed according to the ε-greedy policy. The choice of SARSA is appropriate in stochastic environments like FrozenLake, as the update incorporates its own exploration. An ε-greedy exploration scheme with gradual decay (from 1.0 down to 0.01) has been implemented, so the agent starts exploring and progressively shifts to exploiting what it has learned to obtain more reward. A maximum of 100 steps per episode and 15000 total episodes were also defined to ensure enough experience. The Q-table is initialized to zero and the update exactly follows the SARSA formula using the next action selected by the same policy.

4.1.2 Formulas

The SARSA update follows the following formula:

$$Q(s,a) \leftarrow Q(s,a) + \alpha\left[r + \gamma Q(s',a') - Q(s,a)\right]$$

where s is the current state, a the taken action, r the received reward, s' the next state and a' the new action selected with the same ε-greedy policy. In the code, the calculation is done via:

$$\text{TD target} = r + \gamma Q(s',a')$$
$$Q(s,a) \leftarrow Q(s,a) + \alpha\left(\text{TD target} - Q(s,a)\right)$$

This update is on-policy because it uses the action a' that the agent will actually take according to the exploration policy.

4.1.3 Studied parameters (α, γ, ε)

The three main parameters analyzed are:

- **Learning rate** α: controls to what extent new values replace old ones. Too large values can make the process unstable, while too small values can make convergence very slow.
- **Discount factor** γ: determines the importance of future rewards. In FrozenLake, a high value favors finding paths that lead to the goal even if they are long.
- **Exploration parameter** ε: regulates the probability of choosing random actions. A decaying ε from 1.0 to 0.01 with a constant reduction factor is used, allowing intense initial exploration and stable final exploitation.

The values used during training are: 15000 episodes, a maximum of 100 steps per episode, and an exploration decay of 0.999.

4.1.4 Pros and cons

Pros

- It is an on-policy method, which allows learning a policy consistent with its own exploration.
- Performs well in stochastic environments like FrozenLake with slippery terrain.
- Has a simple implementation and the Q-table is computationally efficient.
- The decay of ε allows a good combination of initial exploration and final exploitation.

Cons

- Convergence can be slower than Q-Learning since it uses the exploratory action a'.
- It is sensitive to hyperparameters (α, γ, ε), which can greatly affect the result; we will see this in more detail in the results section.
- Does not guarantee a global optimal policy, but rather a safe policy consistent with its own exploration.

4.2 Q-Learning

4.2.1 Differences with SARSA

The main difference between Q-Learning and SARSA is that Q-Learning is an off-policy method, while SARSA is on-policy. In SARSA, the update uses the action a' that the agent actually executes according to the ε-greedy policy. Conversely, Q-Learning always uses the value $\max_{a'} Q(s', a')$, regardless of the action it would take with the current policy. This makes Q-Learning more optimistic and aggressive, as it assumes that the optimal action will always be taken in the future. In practice, in stochastic environments like FrozenLake, this can result in lower performance, since the slippery environment breaks this assumption.

4.2.2 Off-policy strategy

Q-Learning updates the Q-table using an off-policy strategy. The update is based on the best possible action in the next state, regardless of the action the agent would take according to its exploration policy. The update rule is:

$$Q(s, a) \leftarrow Q(s, a) + \alpha \left[r + \gamma \max_{a'} Q(s', a') - Q(s, a) \right]$$

This strategy is off-policy because the update assumes optimal future behavior, even if during training the agent follows an ε-greedy policy. This allows exploration independent of the optimal policy but can be less stable in stochastic environments.

4.2.3 Parameters and decisions

The Q-Learning agent uses the same basic hyperparameters as SARSA to maintain experimental consistency: 15000 episodes, a maximum of 100 steps per episode, and an ε-greedy exploration scheme with decay. The three main parameters are:

- **Learning rate** α: controls the magnitude of the update. Larger values accelerate learning but can cause instability.
- **Discount factor** γ: determines the importance of future rewards. With high γ, the agent tends to seek long paths toward the goal.
- **Exploration parameter** ε: initially 1.0, decays to 0.01 with a factor of 0.999. Despite being an off-policy algorithm, Q-Learning needs exploration during training to visit enough states.

The key difference is that, in Q-Learning, the update uses $\max_{a'} Q(s', a')$ instead of the action that would actually be taken. This makes the agent learn a more optimal policy in deterministic environments, but it can be less robust in slippery environments.

4.2.4 Pros and cons

Pros

- Converges toward the optimal policy in deterministic or slightly stochastic environments.
- Has a simple update rule and is widely used in classic RL.
- Does not depend on the action chosen during exploration.
- The final policy is directly greedy with respect to the Q-table.

Cons

- In stochastic environments like slippery FrozenLake, Q-Learning can be too optimistic.
- Performance can be lower than that of SARSA.
- Very sensitive to parameters and may take time to stabilize.
- Can overestimate actions in environments with random transitions.

4.3 Monte Carlo

4.3.1 Full episodes

The Monte-Carlo Control algorithm learns exclusively from full episodes. This means the agent generates a complete trajectory following an ε-greedy policy until reaching a terminal state, and only then calculates the Return G for each (s, a) pair. This return is obtained by accumulating all future rewards from the end to the beginning. In the implementation, First-Visit Monte-Carlo is used, so each (s, a) pair is only updated the first time it appears in an episode. Only after completing the entire episode is the Q-table updated.

4.3.2 Problems in stochastic environments

In stochastic environments like FrozenLake with slippery terrain, Monte-Carlo can struggle to converge, as it depends on the full return G, which can vary greatly due to the random nature of transitions. Not updating step-by-step, but only at the end of each episode, means the agent needs many episodes to get a stable estimate of $Q(s, a)$. Moreover, since the reward is only positive upon reaching the goal, most episodes have a return $G = 0$, making learning particularly slow.

4.3.3 Design decisions

Several decisions have been made to adapt Monte-Carlo to FrozenLake. First, 15000 episodes and a maximum of 100 steps per episode have been used, since Monte-Carlo needs much more sampling than other methods like the two previously seen. An ε-greedy exploration scheme has also been implemented with a slower decay than in SARSA or Q-Learning, using a factor of 0.9997. This is necessary because Monte-Carlo requires extensive exploration of infrequent states. The Q-table update is done with First-Visit: the returns G of each (s, a) pair are stored, and the value $Q(s, a)$ is the average of all observed returns. This guarantees a policy that gradually improves with experience.

4.3.4 Pros and cons

Pros

- Does not need to know the transition model of the environment.
- Is conceptually simple and relies solely on real sampling of executions.
- Guarantees convergence to the optimal policy under sufficient exploration.
- The clear separation between episode generation and updating facilitates analysis.

Cons

- Requires many more episodes than SARSA or Q-Learning to converge.
- Is very slow in environments with sparse rewards like FrozenLake.
- The return G can be highly variable in stochastic environments.
- Does not update step-by-step, so the policy may take time to react to errors.

4.4 Genetic Algorithm

4.4.1 Individual

In this genetic algorithm, each individual represents a complete policy for FrozenLake. Specifically, an individual is an integer vector of size n_states, where each position indicates the action the agent will take in that state. Thus, an individual is of the type:

$$\pi = [a\_0, a\_1, \ldots, a\_n - 1],$$

where each $a\_i \in \{0, 1, \ldots, n\_actions - 1\}$. This encoding makes each individual directly a deterministic policy, which greatly simplifies the evaluation process. The initial population is generated randomly, with each state assigned a uniformly chosen action.

4.4.2 Fitness

The fitness function measures the quality of an individual by evaluating its policy over several episodes. In our case, fitness is the average reward obtained over a set of *episodes_eval* episodes:

$$\text{fitness}(\pi) = \frac{1}{K} \sum_{i=1}^{K} R_i(\pi)$$

where $K = episodes\_eval$. For each episode, the agent follows the actions defined by the policy without exploration, and the obtained reward is accumulated. A high fitness indicates that the policy leads the agent consistently toward the goal. This function is implemented in the `evaluate_policy()` method of the code.

4.4.3 Mutation and crossover

To generate new offspring, two genetic operators are used. The crossover is uniform: for each state, the child inherits the action from *parent1* or *parent2* with a probability of 0.5. This process combines partially good policies with the aim of creating more solid solutions.

Mutation is applied to each position of the vector with a probability of *mutation_rate*. If it occurs, the action in that state is replaced by a random action among the possible ones. This operation introduces diversity and prevents the population from converging prematurely to a local optimum. Both operators are implemented in the `crossover()` and `mutate()` methods.

4.4.4 Design justification

The design of the genetic algorithm follows classic principles but adapted to the nature of FrozenLake. Representing an individual as a complete policy facilitates direct evaluation without the need for Q-tables. Tournament selection is used because it favors solid individuals without requiring costly sorting. Elitism is added by keeping the best individual of each generation, ensuring that the population quality never decreases.

Mutation controls diversity, essential in a stochastic environment where different policies might perform unequally. Finally, an *early stopping* mechanism has been incorporated: if the best fitness does not improve over a set number of generations, the algorithm halts to avoid unnecessary calculations. This set of decisions allows the genetic algorithm to find competitive policies even if it does not guarantee global optima.

4.5 Results

4.5.1 Tables and graphs

*NOTE: To be able to extract correct information about the behavior of the different algorithms, the same parameter values have been established ($\alpha = 0.5$,

$\gamma = 1.0$, episodes = 15000, steps per episode = 100, initial epsilon = 1.0, minimum epsilon = 0.01, epsilon decay = 0.999) for SARSA, Q-Learning, and Monte Carlo. In the next section (Hyperparameters) we will delve into how the different algorithms behave when we alter the hyperparameters.

4.5.2 Results: SARSA Agent

Below are the results obtained during the training of the SARSA agent over 15,000 episodes.

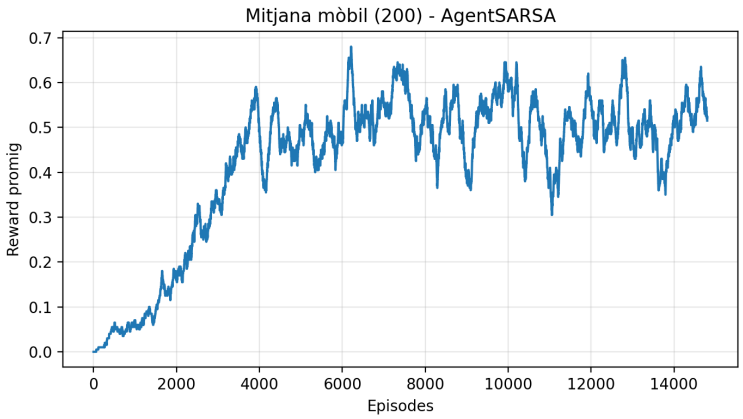

Figure 1: Moving average (200-episode window) of the reward (SARSA). Shows the smoothed learning trend.

Preliminary analysis: As observed in Figure 1, the SARSA agent shows a clear learning curve that begins to grow significantly after the first 1,000 episodes. Policy stabilization (convergence) seems to occur approximately around episode 4,000.

From this point, the average reward fluctuates between 0.45 and 0.65. It is important to note that, due to the stochasticity of the environment (`is_slippery=True`), it is impossible to obtain an average reward of 1.0, since even an optimal policy is subject to accidental falls caused by slipping. The volatility of the graph reflects this inherent uncertainty of the environment and the *on-policy* nature of the algorithm, which learns taking into account the current exploration.

4.5.3 Results: Q-Learning Agent

The results for the agent trained with Q-Learning are presented below. Keep in mind that this is an *off-policy* algorithm, meaning it attempts to learn the value of the optimal policy Q^* regardless of the exploratory actions taken by the agent during training.

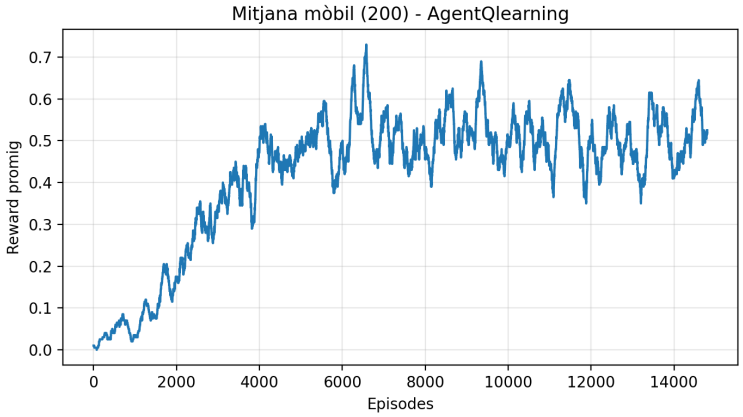

Figure 2: Moving average (200-episode window) of the reward (Q-Learning). Rapid convergence and high performance peaks are observed.

Analysis: As can be seen in Figure 2, the Q-Learning agent shows vigorous learning capacity. The average reward curve starts to ascend rapidly from episode 1,000, showing a steep slope until episode 4,000.

A notable point is the performance peak observed around episode 6,500, where the average reward exceeds the 0.7 threshold. This indicates that the agent has found a highly effective policy to navigate the ice. Subsequently, the graph shows fluctuations between 0.4 and 0.65. This variance does not necessarily imply that the agent "forgets," but rather that in a stochastic environment (`is_slippery=True`), even the mathematically optimal policy has an inherent failure probability that makes the moving average fluctuate. Compared to more conservative methods, Q-Learning seems to better exploit routes that, despite being risky, maximize expected value in the long term.

4.5.4 Results: Monte Carlo Agent

In this execution, the Monte Carlo agent was trained for 15,000 episodes. The following graphs show the evolution of its policy.

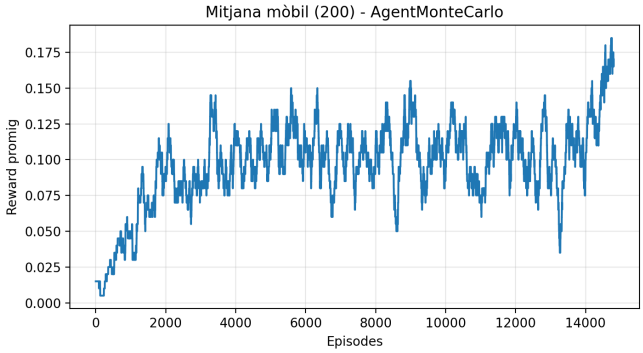

Figure 3: Moving average (window of 200). High instability is observed, along with a final improvement peak reaching 0.18.

Analysis: The data reflects the intrinsic difficulty the Monte Carlo algorithm faces in highly stochastic environments like this one. As shown in Figure 3, the learning curve is very noisy ("sawtooth"), constantly fluctuating between 0.05 and 0.15 for much of the training.

However, an interesting behavior is detected at the end of the experiment (from episode 14,000), where performance spikes up to touch **0.18** (18% success). This indicates the agent was beginning to consolidate a better policy just as the training ended. This slowness and high variance are explained because Monte Carlo does not perform *bootstrapping*: it has to wait until the end of the episode to update values, causing the random "slips" of the map to introduce a lot of noise into the estimation of the actual state values.

4.5.5 Results: Genetic Algorithm

Finally, the results of the evolutionary approach are analyzed. This execution stands out for its extreme efficiency compared to classic Reinforcement Learning methods.

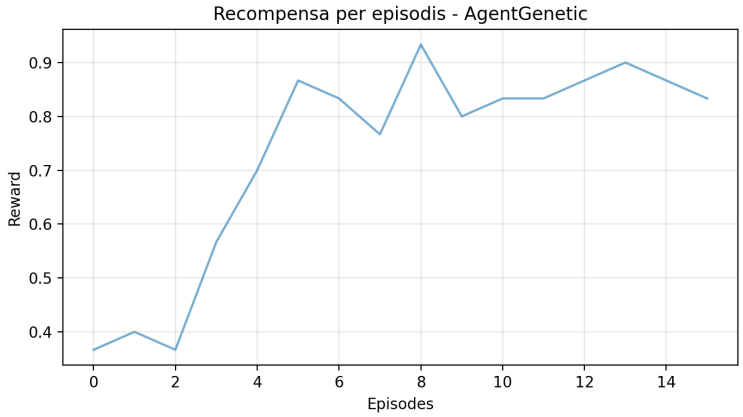

Figure 4: Fitness of the best individual per generation. A drastic improvement in the first 5 generations and a performance peak above 0.9 are observed.

Analysis: As can be seen in Figure 4, the Genetic algorithm is able to find an excellent policy (with a success rate above 90%) in fewer than 15 generations. The curve shows an initial vertical ascent, moving from mediocre performance (0.35) to a very robust one (0.85) in just 5 iterations.

This behavior demonstrates the power of the *crossover* operator to combine good action sequences rapidly. The fact that the graph stops at generation 15 suggests the algorithm has converged quickly to a highly stable optimal or sub-optimal solution, rendering further computations unnecessary. In terms of computation time vs. solution quality, this has been the most efficient algorithm of the assignment.

4.5.6 Analysis of the Resulting Policies

In addition to the numerical metrics, it is fundamental to qualitatively analyze the behavior learned by the agents. Below are the "average" or most representative policies obtained for each algorithm after the 8 executions.

Remember that, due to the `is_slippery=True` property, an "Up" (↑) action does not always move the agent upward. Often, agents learn to "crash" into walls (move toward a blocked direction) to reduce uncertainty and take advantage of the lateral slipping probability to move more safely.

Interpretation of strategies:

1. **The wall strategy (Q-Learning):** If we look at the first row of the Q-Learning agent (S, ↑, ↑, ↑), we see the agent tries to move upwards repeatedly, even though there is a wall. This is a very smart strategy in FrozenLake: if the agent is at the upper edge and moves up, most times it will stay in place, but the "slip" will move it left or right. This allows it

SARSA (Conservative Policy)

S	↑	↓	↑
←	**H**	→	**H**
↑	↓	←	**H**
H	→	↓	**G**

Q-Learning (Greedy Policy)

S	↑	↑	↑
←	**H**	→	**H**
↑	↓	←	**H**
H	→	↓	**G**

Monte Carlo (Unstable Policy)

S	↑	←	↑
←	**H**	→	**H**
↑	↓	↓	**H**
H	→	↓	**G**

Genetic Agent (Optimal Policy)

S	↑	←	↓
←	**H**	→	**H**
↑	↓	←	**H**
H	→	↓	**G**

Table 1: Representation of the learned actions on the 4x4 grid. The arrows indicate the direction of the action with the highest Q value (or genetic) for each state.

to navigate horizontally without the risk of moving down (where the holes in the second row are). It is a strategy that prioritizes long-term safety.

2. **Differences in the danger zone (Row 3, Column 2):** The tile (2,2) is critical because it has a hole to its right and a hole down to its left.
   - **SARSA and Genetic** choose to move left (←). This seems counter-intuitive (they move away from the goal), but it is the safest option to avoid falling into the right hole (H) due to a slip.
   - **Monte Carlo** chooses to move down (↓). This action is very risky in this position and reflects the less precise nature of this algorithm in intermediate states.
3. **The consistency of the Genetic Agent:** The Genetic Agent's policy (which achieved an 82% success rate) shows a very defined route. Unlike the others, at the upper-right corner (0,3) it decides to go straight down (↓). This indicates it has found a specific sequence of actions that, combined with the luck of its high fitness, maximizes the probability of reaching G quickly, avoiding the infinite loops sometimes caused by the overly conservative policies of Q-Learning.

4.6 Hyperparameters

In this section, the main hyperparameters of each algorithm have been analyzed, observing their effects on the convergence rate, stability, and overall

performance.

SARSA. The main hyperparameters are α, γ, and ε. Values of α between 0.4 and 0.6 offer stability; $\alpha > 0.7$ causes oscillations. A high γ $(0.95 - 1.0)$ is necessary to plan for the long term. Using a decaying ε (from 1.0 to 0.01) is essential because SARSA, as an on-policy method, needs to balance exploration and exploitation.

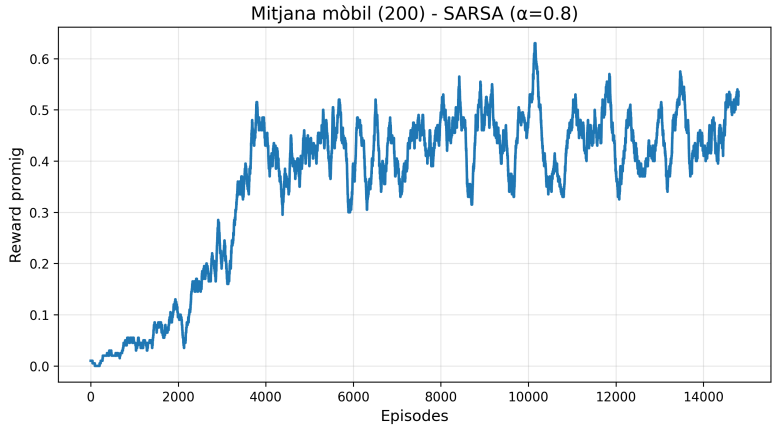

Figure 5: Fitness of the best individual per generation. A drastic improvement is observed during the first generations and a performance peak above 0.9, indicating quick and effective convergence of the genetic algorithm.

As we can observe, now that α is greater than in figure 2, we observe major oscillations, an expected result.

Q-Learning. Despite having the same hyperparameters as SARSA, Q-Learning is more sensitive. An excessively high α can cause overestimations; the optimal range is $[0.3, 0.5]$. A γ close to 1.0 is critical to maximize future reward. An ε too low at the end of the decay yields overly optimistic policies; the recommended range is $\varepsilon \approx 0.05 - 0.15$.

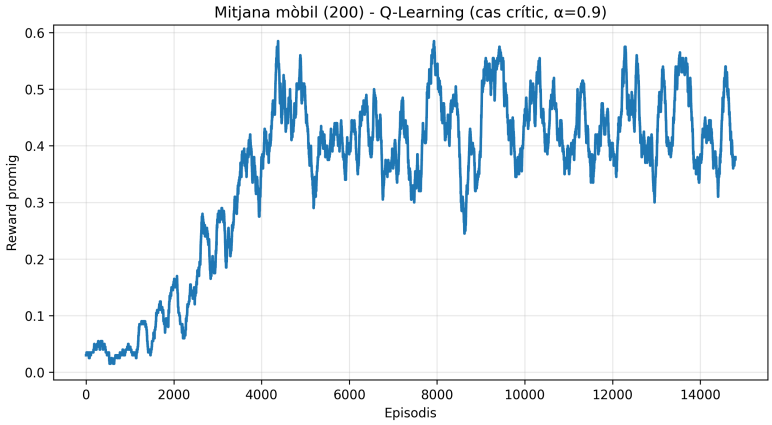

Figure 6: Q-Learning in a critical case ($\alpha = 0.9$). Severe oscillations and a lack of convergence are shown due to its off-policy nature and overly aggressive learning.

Monte Carlo. The determining factor is the number of full episodes. With fewer than 5000 episodes, the results are inconsistent; from 15000 episodes onward, the variance is reduced. A sufficiently high ε-greedy is also essential to explore entire trajectories to the goal.

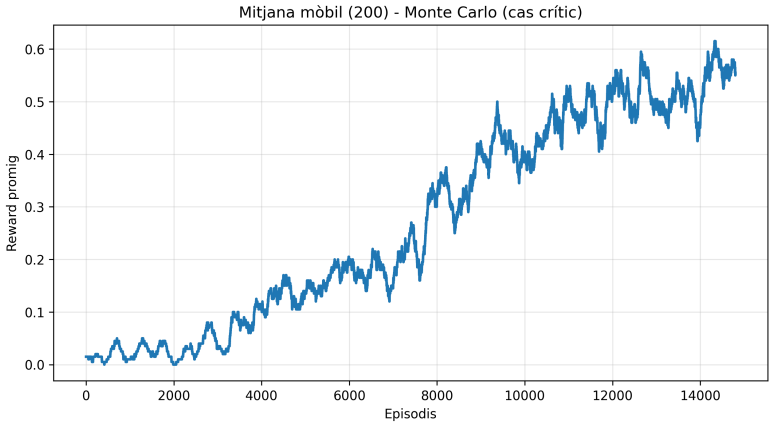

Figure 7: Since Monte Carlo with a critical case, $\gamma = 1$, does not update until the episode ends, it is consistent that a high percentage is not reached until the end.

Genetic algorithm. The hyperparameters are population, generations, and mutation rate. Small populations (< 20) converge prematurely; values between 40 and 60 work better. Clear improvements are obtained up to 80–120 genera-

tions. The optimal mutation is between 0.08 and 0.15: too low loses diversity and too high destroys useful information. Early stopping reduces training time when there is no improvement.

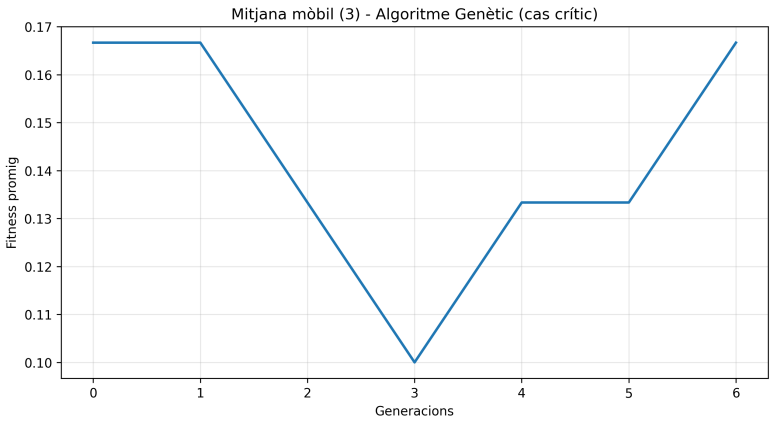

Figure 8: Genetic algorithm in a critical case. The combination of an overly small population, high mutation, and few generations leads to extremely unstable learning without convergence.

4.7 Global performance comparison

To ensure the scientific validity of the conclusions and mitigate the effect of stochasticity (especially relevant in the Monte Carlo algorithm), each agent was executed **5 times** independently. Table 2 gathers the means of the success rate (the last 500 episodes), the standard deviation, and the computation time.

Algorithm	**Mean Success (μ)**	**Deviation (σ)**	**Time (s)**	**Time (σ)**
SARSA	0.4936	$\pm$0.0407	8.68 s	$\pm$0.11
Q-Learning	0.4872	$\pm$0.0298	13.59 s	$\pm$2.28
Monte Carlo	0.3772	$\pm$**0.1835**	52.45 s	$\pm$16.95
Genetic Agent	**0.8225**	$\pm$**0.0052**	17.84 s	$\pm$0.44

Table 2: Aggregated results of 5 independent executions (25,000 episodes for RL, 80 generations for Genetic).

Performance Analysis:

1. **Dominance of the Genetic algorithm:** The data shows an indisputable victory for the evolutionary approach. The Genetic Agent has achieved a success rate of **82.25%**, well above any RL method (which stagnate around 49%).

Moreover, it is the most robust algorithm, with an insignificant standard deviation ($\sigma = 0.005$). This indicates that, regardless of the random seed, the genetic algorithm always finds an excellent solution. As observed in Figure 9 (red line), the algorithm reaches this maximum performance in fewer than 20 generations, demonstrating learning efficiency vastly superior to the thousands of episodes needed by the other methods.

2. **The instability of Monte Carlo:** The results theoretically and practically confirm Monte Carlo's weakness in high-variance environments. It is the algorithm with the worst mean performance (**37.7%**) and, most importantly, with a giant standard deviation ($\sigma = 0.183$). This means its behavior is unpredictable: in some executions it might work moderately well, but in others it fails miserably. Furthermore, it proved to be the slowest method by far (52.45s), almost 6 times slower than SARSA, due to the memory management of the full episode traces.

3. **SARSA vs Q-Learning:** Both Temporal Difference (TD) algorithms show almost identical performance in terms of success ($\sim 49\%$). However, **SARSA** proved to be more computationally efficient in this test battery, taking an average of 8.68 seconds compared to Q-Learning's 13.59 seconds. This suggests that, for this specific problem, SARSA's conservative policy is as effective as Q-Learning's *greedy* one, but it converges with less cost.

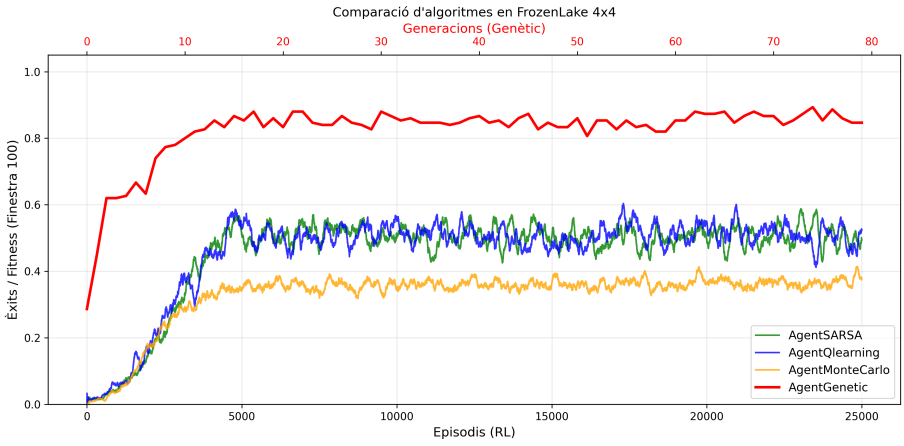

Figure 9: Visual comparison. The upper axis (red) corresponds to the Genetic's generations, while the lower to the RL episodes. Note the superiority and stability of the red curve.

Conclusion: For the *FrozenLake* environment with slippery terrain, the **Genetic algorithm** is, without a doubt, the best available option, combining

maximum performance with total stability. Among the classic Reinforcement Learning options, **SARSA** offers the best balance between execution time and results.

5 Use of generative artificial intelligence

In accordance with the assignment guidelines, the use of the following generative artificial intelligence tools during the preparation of this work is declared:

- **Tool:** ChatGPT (Model 5.1)
- **Purpose:** Support in understanding code errors and specific queries about Python syntax. Furthermore, once we identified the failing algorithm (Dynamic Programming, which requires knowing the complete environment model), the tool provided us with the "unwrapped" property to solve the problem. It also provided an algorithm for the main class that ran multiple executions of the different algorithms to draw more accurate conclusions.
- **Tool:** Gemini (Model 3.0)
- **Purpose:** Assistance in drafting and structuring the report in LaTeX, generating scripts for the visualization of comparative graphs (matplotlib), as well as inserting them into the document. Finally, style and grammar review of the document.